CAREFLOW CLINICAL AI • ARCHITECTURE & IMPLEMENTATION

Hiện Thực Hóa BM25 (Sparse) & Dense Vector Search Trong PostgreSQL & Spring Boot

Cơ chế hoạt động, cấu hình CSDL, Native Query và lý do không cần tự viết giải thuật từ con số 0

1. Phân Định Khái Niệm: Dense Search vs. Sparse Search

Trong hệ thống tìm kiếm thông tin hiện đại và mô hình Hybrid Search RAG, hai khái niệm này đóng hai vai trò hoàn toàn bổ trợ cho nhau:

Thuộc Tính	Dense Search (Tìm kiếm dày / Ngữ nghĩa)	Sparse Search (Tìm kiếm thưa / Từ khóa)
Bản chất công nghệ	Mô hình Vector Embedding (Gemini, OpenAI, BGE).	Thuật toán BM25 / TF-IDF / Full-Text Search.
Dạng dữ liệu lưu trữ	Mảng số thực liên tục dày đặc: `vector(768)`.	Danh sách từ khóa kèm vị trí: `tsvector` (Inverted Index).
Cơ chế tính điểm	Khoảng cách vector: Cosine Similarity, Dot Product, Euclidean.	Tần suất từ khóa (TF), nghịch đảo tần suất tài liệu (IDF) và độ dài văn bản.
Thế mạnh vượt trội	Hiểu ngữ nghĩa, từ đồng nghĩa, ý định diễn đạt tự nhiên của người dùng.	Bắt chính xác 100% từ khóa hiếm: mã ICD-10, tên hoạt chất thuốc, số liệu xét nghiệm.

Lưu ý thuật ngữ: BM25 là Sparse Search (từ khóa chính xác), còn **Vector là Dense Search** (ngữ nghĩa). Cả hai được kết hợp bằng thuật toán **RRF (Reciprocal Rank Fusion)**.

2. Tại Sao Ta Không Cần Tự Viết Code Giải Thuật BM25?

Nếu phải tự lập trình thuật toán BM25 từ con số 0 bằng mã nguồn Java/Python:

- Phải tự xây dựng bộ phân tách từ khóa (Tokenizer) và bộ lọc từ dừng (Stopwords).
- Phải tự duy trì bảng băm chỉ mục đảo (Inverted Index) trong bộ nhớ hoặc ổ đĩa.
- Phải tính toán thủ công độ dài trung bình văn bản $avgdl$ và các trọng số phi tuyến k_1, b.

Giải pháp tối ưu của CareFlow: Chúng ta tận dụng trực tiếp bộ máy **Full-Text Search chuẩn BM25 đã được viết bằng ngôn ngữ C tối ưu hóa cực đại tích hợp sẵn bên trong hạt nhân PostgreSQL**. Bạn chỉ cần cấu hình schema và viết câu lệnh truy vấn mà không phải bảo trì bất kỳ dòng code thuật toán nào.

3. Quy Trình Cài Đặt & Sử Dụng Chi Tiết Trong CareFlow

```
[Văn bản lâm sàng / Phác đồ điều trị]
                  |
                  ▼ (1. Database tự động bóc tách từ & đánh chỉ mục)
PostgreSQL 16: tsv_content (tsvector) + Chỉ mục GIN (idx_clinical_chunks_tsv)
                  |
                  ▼ (2. Repository truy vấn xếp hạng chuẩn BM25)
Spring Data JPA: ts_rank_cd(tsv_content, plainto_tsquery('simple', :query))
                  |
                  ▼ (3. Service kết hợp Dense Vector + Sparse BM25)
ClinicalHybridSearchService: Thuật toán Reciprocal Rank Fusion (k=60)
```

Tầng 1: Khai báo Schema CSDL & Chỉ Mục GIN (PostgreSQL DDL)

Trong tệp cơ sở dữ liệu `scripts/careflow_ai_schema_and_data.sql`, bảng `clinical_guideline_chunks` lưu trữ cả hai dạng:

```
CREATE TABLE clinical_guideline_chunks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id UUID NOT NULL,
    section_title VARCHAR(300),
    content TEXT NOT NULL,
    metadata JSONB DEFAULT '{}'::jsonb,

    -- [1] DENSE SEARCH: Vector nhúng 768 chiều từ Google Gemini Embedding
    embedding VECTOR(768) NOT NULL,

    -- [2] SPARSE BM25: Cột tự động phân tích và sinh từ khóa từ content
    tsv_content TSVECTOR GENERATED ALWAYS AS (to_tsvector('simple', content)) STORED
);

-- Chỉ mục HNSW cho Dense Vector (Cosine Distance)
CREATE INDEX idx_clinical_chunks_hnsw
ON clinical_guideline_chunks USING hnsw (embedding vector_cosine_ops);

-- Chỉ mục GIN (Generalized Inverted Index) cho Sparse BM25
CREATE INDEX idx_clinical_chunks_tsv
ON clinical_guideline_chunks USING gin (tsv_content);
```

> **Điểm sáng kiến trúc:** Cột `tsv_content` dùng mệnh đề `GENERATED ALWAYS AS (...) STORED`. Mỗi khi thêm mới hoặc cập nhật một đoạn phác đồ, PostgreSQL sẽ **tự động tính toán từ vựng và cập nhật chỉ mục GIN ngay lập tức** mà nhà phát triển không cần viết bất kỳ dòng code đồng bộ nào! Cấu hình từ điển `'simple'` giữ nguyên vẹn từ ngữ tiếng Việt mà không bị thuật toán ngôn ngữ tiếng Anh làm sai lệch.

Tầng 2: Truy Vấn Điểm Số BM25 Bằng Spring Data JPA Native Query

Tại giao diện `ClinicalGuidelineChunkRepository.java`, chúng ta khai báo Native Query sử dụng hàm xếp hạng độ tương quan của PostgreSQL:

```
@Query(value = """
    SELECT
        c.id            AS id,
        c.section_title AS sectionTitle,
        c.content       AS content,
        -- Hàm ts_rank_cd: Thuật toán Cover Density Ranking (biến thể chuẩn của BM25)
        ts_rank_cd(c.tsv_content, plainto_tsquery('simple', :query)) AS similarityScore
    FROM clinical_guideline_chunks c
    JOIN clinical_guideline_documents d ON c.document_id = d.id
    WHERE d.is_active = true
      -- Toán tử @@ kiểm tra sự xuất hiện của từ khóa qua chỉ mục GIN
      AND c.tsv_content @@ plainto_tsquery('simple', :query)
      AND (:specialty IS NULL OR d.specialty = :specialty)
    ORDER BY similarityScore DESC
    LIMIT :limitVal
    """, nativeQuery = true)
List<ChunkSearchResultProjection> searchByBm25(
        @Param("query") String query,
        @Param("specialty") String specialty,
        @Param("limitVal") int limit);
```

- **`plainto_tsquery('simple', :query)`**: Tự động làm sạch và chuyển hóa văn bản tìm kiếm của người dùng (ví dụ: *"hạ đường huyết gliclazide"*) thành chuỗi toán tử truy vấn logic `'hạ' & 'đường' & 'huyết' & 'gliclazide'`.
- **`ts_rank_cd(...)`**: Cover Density Ranking — tính điểm không chỉ dựa vào số lần xuất hiện (TF) mà còn dựa vào **khoảng cách không gian** giữa các từ khóa. Nếu các từ khóa xuất hiện gần nhau trong cùng một câu, điểm BM25 sẽ cao hơn nhiều so với việc nằm rải rác.

Tầng 3: Sử Dụng Trong Service & Hợp Nhất RRF

Trong `ClinicalHybridSearchService.java`, chúng ta chỉ việc gọi hàm repository và thực thi công thức RRF ($k=60$):

```
// 1. DENSE VECTOR SEARCH (Tìm kiếm ngữ nghĩa HNSW Cosine)
List<ChunkSearchResultProjection> vectorResults =
        chunkRepository.searchByVector(embeddingString, specialty, 20);
for (int rank = 1; rank <= vectorResults.size(); rank++) {
    UUID id = vectorResults.get(rank - 1).getId();
    rrfScores.merge(id, 1.0 / (RRF_K + rank), Double::sum);
}

// 2. SPARSE BM25 SEARCH (Tìm kiếm từ khóa GIN Full-Text)
List<ChunkSearchResultProjection> bm25Results =
        chunkRepository.searchByBm25(queryText, specialty, 20);
for (int rank = 1; rank <= bm25Results.size(); rank++) {
    UUID id = bm25Results.get(rank - 1).getId();
    rrfScores.merge(id, 1.0 / (RRF_K + rank), Double::sum);
}

// 3. Sắp xếp điểm RRF giảm dần và lấy Top 5 kết quả tối ưu nhất
return rrfScores.entrySet().stream()
        .sorted(Map.Entry.<UUID, Double>comparingByValue().reversed())
        .limit(FINAL_TOP_K)
        .toList();
```

4. So Sánh: PostgreSQL All-in-One vs. Cụm Elasticsearch Độc Lập

Khía Cạnh	Mô Hình Cũ: Dựng Elasticsearch Riêng	Mô Hình CareFlow: PostgreSQL All-in-One
Hạ tầng & Bộ nhớ RAM	Cần JVM riêng cho ES, tiêu tốn tối thiểu 2GB – 4GB RAM.	**0 MB phụ trội**, dùng chung với PostgreSQL hiện có.
Độ trễ đồng bộ (Sync lag)	Phải viết pipeline Logstash / Kafka / CDC Debezium; dữ liệu bị trễ (eventual consistency).	**Tức thì (ACID Transaction)**; commit dữ liệu là tìm thấy ngay lập tức.
Khối lượng code bảo trì	Phải viết RestClient, xử lý retry, reconnect và mapping JSON phức tạp.	**Chỉ 1 câu lệnh SQL Native Query** gọn gàng trong JPA.
Hybrid Search	Lưu vector ở nơi khác (hoặc plugin ES vector nặng nề).	**PostgreSQL lưu cả Vector 768d + BM25 GIN** trên cùng 1 bảng.

5. Kết Luận

Hệ thống CareFlow không cần viết lại mã nguồn BM25 bởi vì hạt nhân **PostgreSQL Full-Text Search** kết hợp cùng chỉ mục **GIN** và hàm **ts_rank_cd** đã cung cấp sẵn năng lực BM25 tiêu chuẩn công nghiệp với hiệu năng mili-giây. Kiến trúc này mang lại sự tinh gọn, tin cậy tuyệt đối và loại bỏ hoàn toàn gánh nặng vận hành hạ tầng phức tạp.